

GD:DI 식별 hvv o 알문상 문상 알세약 바알 문사 알응세문은 속문알에
속세상 바알 문상 알세약 바알 문사 알응세문은 속문알에

lr 5b 군장 개전 개개 팔
lv vv .?A3 5AB7E4AB: E
l _ 과 5mm

a 과 개 개 4 과 5mm 3 a 과 개 개 4 과 5mm 5_ 과 5mm 5
a 과 개 개 4 과 5mm /가 개 개 개 0 5 5
a 과 개 개 4 과 5mm | `남 개 개 군 ' 5
a 과 개 개 4 과 5mm `남 개 개 군 5

GD:DI 식별 hvv

_ 과 5mm |
rr_
u 개 개 개 개
4 |
빠르고 작음 느리고 큼
메모리 (GPU/RAM) ← 디스크 (SSD)
↑ ↑
핫 데이터 콜드 데이터
(최근 접근) (대부분)
_ 과 5mm |
|
lrr_ qSl
|

GD:E

|
o 3 u 간 와 u 극 i 5

문제: FILTERED_KNN(q, K, P)
입력: 쿼리 q, K, 필터 P
출력: {v ∈ V | P(v)} 중 q에 가장 가까운 K개
도전:
1. 필터 조건을 만족하는 벡터의 개수 미지수
2. 필터를 만족하는 벡터가 매우 드물 수 있음
3. 필터 조건이 벡터 공간의 구조와 무관

|

선택도 = $\{V_P\} / \{V\}$

선택도 99% (거의 모든 벡터 선택):

- 기본 ANN 검색과 거의 동일
- 작은 오버헤드

선택도 1% (대부분 필터링 제외):

- 넓은 검색 범위 필요
- 높은 계산 비용

선택도 0.1% (매우 선택적):

- 극도로 넓은 검색 필요
- 거의 선형 탐색 수준

GD:F n 선택도 범위 식별 hv v

4 |

: 5 4 /a 광대역 근거리

개

: 5

?5 i mm

|

|

개

?5 /c 근거리

개

: 5

?5

A5

|

|

개

GD:G n 선택도 범위 식별 hv v

|

Vamana 그래프:

- 각 벡터가 노드
- 각 노드는 M개의 이웃 연결
- 거리 기반의 엣지 가중치

메타데이터:

- 각 벡터의 속성값 저장
- 속성 인덱스 (속성 값 → 벡터 ID)
- 선택도 통계

|

```
procedure FILTERED_SEARCH(q, K, filter_predicate,
                           num_search_nodes):
    // 1. 진입점 선택 (필터 만족하는 점)
    entry_point = find_entry_point(filter_predicate)

    // 2. 초기 후보 수집
    visited = {}
    candidates = priority_queue() // 거리순 정렬
    result_set = {}

    candidates.push(entry_point)
    visited.add(entry_point)

    // 3. 그리디 검색
```

```

while not candidates.empty() and
    |visited| < num_search_nodes:

    current = candidates.pop()

    // 필터 조건 확인
    if not filter_predicate(current):
        continue // 필터 미충족 스킵

    // 결과 집합 추가
    dist = distance(q, current)
    if |result_set| < K or dist < max_dist(result_set):
        result_set.add((current, dist))
        if |result_set| > K:
            remove_farthest(result_set)

    // 이웃 탐색
    for neighbor in neighbors[current]:
        if neighbor not in visited:
            visited.add(neighbor)
            candidates.push(neighbor)

// 4. 결과 정렬 및 반환
return sort_by_distance(result_set)

```

GD:I

|

필터 선택도 추정:

```
selectivity = |결과_필터_통과| / |방문_노드|
```

검색 범위 조정:

```

if selectivity > 50%: // 필터 선택적이지 않음
    num_search_nodes = K * 2
elif selectivity > 10%:
    num_search_nodes = K * 5
elif selectivity > 1%:
    num_search_nodes = K * 20
else:
    num_search_nodes = K * 100 또는 더 큼

```

GD:N

|

속성별 비트맵 인덱스:

```

color="red" → 비트맵 001010...
(해당 벡터는 1, 아니면 0)

```

장점: 빠른 필터 평가

단점: 메모리 사용량 (속성당 1 비트/벡터)

|

B-트리 기반 속성 인덱스:

```

범위 필터 (price > 100) 지원
효율적인 범위 쿼리

```

디스크 I/O 최적화:

```
자주 함께 필터링되는 속성들을 근처에 배치
```

GD:O

|

필터를 만족하는 진입점을 미리 식별:

방법 1: 필터별 진입점 캐싱

```
filtered_entry_point[filter_id] = p
```

방법 2: 범용 진입점에서 필터 만족점까지 빠른 이동
entry_point에서 시작
→ 이웃 탐색으로 필터 만족점 빠르게 찾기

|
필터 친화적 이웃 선택:

전통적 Vamana:
각 점 p의 이웃 = 가장 가까운 M개

필터 인식 Vamana:
각 점 p의 이웃 = 거리/필터_상관성 혼합 기반
- 같은 필터 그룹의 점들을 함께 연결
- 그룹 간 "다리" 역할 노드 유지

GD:[qw

|
메모리 계층:

- 1단계: GPU 캐시 (가장 자주 접근)
- 2단계: RAM 버퍼 (핫 노드)
- 3단계: SSD 캐시 (온도 낮은 노드)
- 4단계: 메인 SSD (대부분의 데이터)

dn|
여러 쿼리의 필요한 벡터들을 함께 로드:
쿼리 1 → 노드 {1, 5, 12, 3}
쿼리 2 → 노드 {2, 5, 8, 3}
쿼리 3 → 노드 {1, 7, 12, 8}

공통 노드 {1, 5, 12, 3, 2, 8, 7}를 한 번에 로드

GD:]

|
벡터 데이터: $N \times dim \times 4$ 바이트 (SSD에 저장)
메타그래프: $N \times M \times 4$ 바이트 (메모리, $M=64$)
= 1M 벡터, $M=64$: 256MB
속성 인덱스: 속성 수 $\times$ (인덱스 구조 크기)
= 100개 속성, 단순 비트맵: 12.5MB/속성

GD:DA

|
price > 100
color == "red"

|
(price > 100 AND price < 500) AND
(color == "red" OR color == "blue") AND
(stock > 0)

- 평가 순서 최적화:
- 가장 선택적인 조건부터 평가
 - 조기 종료 (AND에서 첫 거짓)
 - 속성 인덱스 활용 (범위 쿼리)

GD:DD an색을볼수있습니v v 용의상대알

	a광대개관 5mm	남개관
능능능능능능능능능능능능능능능능	u거거거거	c m v
		4

	다	

GD:DE m상에알
은모는

|

|

|

V4

|

: 5				P		P
?5			3			
P						
A5cm v		la가게 4 고5mm	cm v /`늘게도	0		P
B5					P	
D5				P		
E5						P
F5				P		